3.6 Interpolation on a Grid in Multidimensions

In multidimensional interpolation, we seek an estimate of a function of more than one independent variable, $y(x_1, x_2, \ldots, x_n)$. The Great Divide is, Are we given a complete set of tabulated values on an n-dimensional grid? Or, do we know function values only on some scattered set of points in the n-dimensional space? In one dimension, the question never arose, because any set of x_i's, once sorted into ascending order, could be viewed as a valid one-dimensional grid (regular spacing not being a requirement).

As the number of dimensions n gets large, maintaining a full grid becomes rapidly impractical, because of the explosion in the number of gridpoints. Methods that work with scattered data, to be considered in §3.7, then become the methods of choice. Don't, however, make the mistake of thinking that such methods are intrinsically more accurate than grid methods. In general they are less accurate. Like the proverbial three-legged dog, they are remarkable because they work at all, not because they work, necessarily, well!

Both kinds of methods are practical in two dimensions, and some other kinds as well. For example, *finite element methods*, of which *triangulation* is the most common, find ways to impose some kind of geometrically regular structure on scattered points, and then use that structure for interpolation. We will treat two-dimensional interpolation by triangulation in detail in §21.6; that section should be considered as a continuation of the discussion here.

In the remainder of this section, we consider only the case of interpolating on a grid, and we implement in code only the (most common) case of two dimensions. All of the methods given generalize to three dimensions in an obvious way. When we implement methods for scattered data, in §3.7, the treatment will be for general n.

In two dimensions, we imagine that we are given a matrix of functional values y_{ij}, with $i = 0, \ldots, M-1$ and $j = 0, \ldots, N-1$. We are also given an array of x_1 values x_{1i}, and an array of x_2 values x_{2j}, with i and j as just stated. The relation of these input quantities to an underlying function $y(x_1, x_2)$ is just

$$y_{ij} = y(x_{1i}, x_{2j}) \tag{3.6.1}$$

We want to estimate, by interpolation, the function y at some untabulated point (x_1, x_2).

An important concept is that of the *grid square* in which the point (x_1, x_2) falls, that is, the four tabulated points that surround the desired interior point. For convenience, we will number these points from 0 to 3, counterclockwise starting from the lower left. More precisely, if

$$\begin{aligned} x_{1i} \le x_1 \le x_{1(i+1)} \\ x_{2j} \le x_2 \le x_{2(j+1)} \end{aligned} \tag{3.6.2}$$

defines values of i and j, then

$$\begin{aligned} y_0 &\equiv y_{ij} \\ y_1 &\equiv y_{(i+1)j} \\ y_2 &\equiv y_{(i+1)(j+1)} \\ y_3 &\equiv y_{i(j+1)} \end{aligned} \tag{3.6.3}$$

The simplest interpolation in two dimensions is *bilinear interpolation* on the grid square. Its formulas are

$$\begin{aligned} t &\equiv (x_1 - x_{1i})/(x_{1(i+1)} - x_{1i}) \\ u &\equiv (x_2 - x_{2j})/(x_{2(j+1)} - x_{2j}) \end{aligned} \tag{3.6.4}$$

(so that t and u each lie between 0 and 1) and

$$y(x_1, x_2) = (1-t)(1-u)y_0 + t(1-u)y_1 + tuy_2 + (1-t)uy_3 \tag{3.6.5}$$

Bilinear interpolation is frequently "close enough for government work." As the interpolating point wanders from grid square to grid square, the interpolated function value changes continuously. However, the gradient of the interpolated function changes discontinuously at the boundaries of each grid square.

We can easily implement an object for bilinear interpolation by grabbing pieces of "machinery" from our one-dimensional interpolation classes:

interp_2d.h

```
struct Bilin_interp {
```

Object for bilinear interpolation on a matrix. Construct with a vector of x_1 values, a vector of x_2 values, and a matrix of tabulated function values y_{ij}. Then call `interp` for interpolated values.

```
    Int m,n;
    const MatDoub &y;
    Linear_interp x1terp, x2terp;

    Bilin_interp(VecDoub_I &x1v, VecDoub_I &x2v, MatDoub_I &ym)
        : m(x1v.size()), n(x2v.size()), y(ym),
        x1terp(x1v,x1v), x2terp(x2v,x2v) {}
```

Construct dummy 1-dim interpolations for their `locate` and `hunt` functions.

```
    Doub interp(Doub x1p, Doub x2p) {
        Int i,j;
        Doub yy, t, u;
        i = x1terp.cor ? x1terp.hunt(x1p) : x1terp.locate(x1p);
        j = x2terp.cor ? x2terp.hunt(x2p) : x2terp.locate(x2p);
```

Find the grid square.

```
        t = (x1p-x1terp.xx[i])/(x1terp.xx[i+1]-x1terp.xx[i]);
        u = (x2p-x2terp.xx[j])/(x2terp.xx[j+1]-x2terp.xx[j]);
        yy = (1.-t)*(1.-u)*y[i][j] + t*(1.-u)*y[i+1][j]
            + (1.-t)*u*y[i][j+1] + t*u*y[i+1][j+1];
        return yy;
    }
};
```

Interpolate.

Here we declare two instances of `Linear_interp`, one for each direction, and use them merely to do the bookkeeping on the arrays x_{1i} and x_{2_j} — in particular, to provide the "intelligent" table-searching mechanisms that we have come to rely on. (The second occurrence of `x1v` and `x2v` in the constructors is just a placeholder; there are not really any one-dimensional "y" arrays.)

Usage of `Bilin_interp` is just what you'd expect:

```
Int m=..., n=...;
MatDoub yy(m,n);
VecDoub x1(m), x2(n);
...
Bilin_interp myfunc(x1,x2,yy);
```

followed (any number of times) by

```
Doub x1,x2,y;
...
y = myfunc.interp(x1,x2);
```

Bilinear interpolation is a good place to start, in two dimensions, unless you positively know that you need something fancier.

There are two distinctly different directions that one can take in going beyond bilinear interpolation to higher-order methods: One can use higher order to obtain increased accuracy for the interpolated function (for sufficiently smooth functions!), without necessarily trying to fix up the continuity of the gradient and higher derivatives. Or, one can make use of higher order to enforce smoothness of some of these derivatives as the interpolating point crosses grid-square boundaries. We will now consider each of these two directions in turn.

3.6.1 Higher Order for Accuracy

The basic idea is to break up the problem into a succession of one-dimensional interpolations. If we want to do `m-1` order interpolation in the x_1 direction, and `n-1` order in the x_2 direction, we first locate an $m \times n$ sub-block of the tabulated function matrix that contains our desired point (x_1, x_2). We then do `m` one-dimensional interpolations in the x_2 direction, i.e., on the rows of the sub-block, to get function values at the points (x_{1i}, x_2), with m values of i. Finally, we do a last interpolation in the x_1 direction to get the answer.

Again using the previous one-dimensional machinery, this can all be coded very concisely as

interp_2d.h

```
struct Poly2D_interp {
```

Object for two-dimensional polynomial interpolation on a matrix. Construct with a vector of x_1 values, a vector of x_2 values, a matrix of tabulated function values y_{ij}, and integers to specify the number of points to use locally in each direction. Then call `interp` for interpolated values.

```
    Int m,n,mm,nn;
    const MatDoub &y;
    VecDoub yv;
    Poly_interp x1terp, x2terp;

    Poly2D_interp(VecDoub_I &x1v, VecDoub_I &x2v, MatDoub_I &ym,
        Int mp, Int np) : m(x1v.size()), n(x2v.size()),
        mm(mp), nn(np), y(ym), yv(m),
        x1terp(x1v,yv,mm), x2terp(x2v,x2v,nn) {}
```

Dummy 1-dim interpolations for their `locate` and `hunt` functions.

```
    Doub interp(Doub x1p, Doub x2p) {
        Int i,j,k;
        i = x1terp.cor ? x1terp.hunt(x1p) : x1terp.locate(x1p);
        j = x2terp.cor ? x2terp.hunt(x2p) : x2terp.locate(x2p);
        Find grid block.
        for (k=i;k<i+mm;k++) {
            x2terp.yy = &y[k][0];
            yv[k] = x2terp.rawinterp(j,x2p);
        }
        return x1terp.rawinterp(i,x1p);
    }
};
```

`mm` interpolations in the x_2 direction.

A final interpolation in the x_1 direction.

The user interface is the same as for `Bilin_interp`, except that the constructor has two additional arguments that specify the number of points (order plus one) to be used locally in, respectively, the x_1 and x_2 interpolations. Typical values will be in the range 3 to 7.

Code stylists won't like some of the details in `Poly2D_interp` (see discussion in §3.1 immediately following `Base_interp`). As we loop over the rows of the sub-block, we reach into the guts of `x2terp` and repoint its `yy` array to a row of our y matrix. Further, we alter the contents of the array `yv`, for which `x1terp` has stored a pointer, on the fly. None of this is particularly dangerous as long as we control the implementations in both `Base_interp` and `Poly2D_interp`; and it makes for a very efficient implementation. You should view these two classes as not just (implicitly) `friend` classes, but as *really intimate* friends.

3.6.2 Higher Order for Smoothness: Bicubic Spline

A favorite technique for obtaining smoothness in two-dimensional interpolation is the *bicubic spline*. To set up a bicubic spline, you (one time) construct M one-dimensional splines across the rows of the two-dimensional matrix of function values. Then, for each desired interpolated value you proceed as follows: (1) Perform M spline interpolations to get a vector of values $y(x_{1i}, x_2), i = 0, \ldots, M-1$. (2) Construct a one-dimensional spline through those values. (3) Finally, spline-interpolate to the desired value $y(x_1, x_2)$.

If this sounds like a lot of work, well, yes, it is. The one-time setup work scales as the table size $M \times N$, while the work per interpolated value scales roughly as $M \log M + N$, both with pretty hefty constants in front. This is the price that you pay for the desirable characteristics of splines that derive from their nonlocality. For tables with modest M and N, less than a few hundred, say, the cost is usually tolerable. If it's not, then fall back to the previous local methods.

Again a very concise implementation is possible:

interp_2d.h

```
struct Spline2D_interp {
```
Object for two-dimensional cubic spline interpolation on a matrix. Construct with a vector of x_1 values, a vector of x_2 values, and a matrix of tabulated function values y_{ij}. Then call `interp` for interpolated values.
```
    Int m,n;
    const MatDoub &y;
    const VecDoub &x1;
    VecDoub yv;
    NRvector<Spline_interp*> srp;

    Spline2D_interp(VecDoub_I &x1v, VecDoub_I &x2v, MatDoub_I &ym)
        : m(x1v.size()), n(x2v.size()), y(ym), yv(m), x1(x1v), srp(m) {
        for (Int i=0;i<m;i++) srp[i] = new Spline_interp(x2v,&y[i][0]);
        Save an array of pointers to 1-dim row splines.
    }

    ~Spline2D_interp(){
        for (Int i=0;i<m;i++) delete srp[i];        We need a destructor to clean up.
    }

    Doub interp(Doub x1p, Doub x2p) {
        for (Int i=0;i<m;i++) yv[i] = (*srp[i]).interp(x2p);
        Interpolate on each row.
        Spline_interp scol(x1,yv);                  Construct the column spline,
        return scol.interp(x1p);                    and evaluate it.
    }
};
```

The reason for that ugly vector of pointers to `Spline_interp` objects is that we need to initialize each row spline separately, with data from the appropriate row. The user interface is the same as `Bilin_interp`, above.

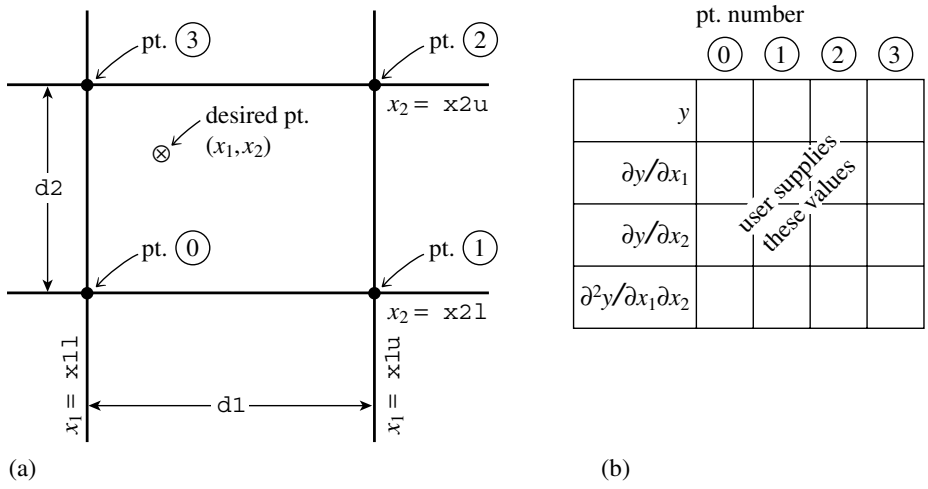

Figure 3.6.1. (a) Labeling of points used in the two-dimensional interpolation routines `bcuint` and `bcucof`. (b) For each of the four points in (a), the user supplies one function value, two first derivatives, and one cross-derivative, a total of 16 numbers.

3.6.3 Higher Order for Smoothness: Bicubic Interpolation

Bicubic interpolation gives the same degree of smoothness as bicubic spline interpolation, but it has the advantage of being a local method. Thus, after you set it up, a function interpolation costs only a constant, plus $\log M + \log N$, to find your place in the table. Unfortunately, this advantage comes with a lot of complexity in coding. Here, we will give only some building blocks for the method, not a complete user interface.

Bicubic splines are in fact a special case of bicubic interpolation. In the general case, however, we leave the values of all derivatives at the grid points as freely specifiable. You, the user, can specify them *any way you want*. In other words, you specify at each grid point not just the function $y(x_1, x_2)$, but also the gradients $\partial y/\partial x_1 \equiv y_{,1}$, $\partial y/\partial x_2 \equiv y_{,2}$ and the cross derivative $\partial^2 y/\partial x_1 \partial x_2 \equiv y_{,12}$ (see Figure 3.6.1). Then an interpolating function that is *cubic* in the scaled coordinates t and u (equation 3.6.4) can be found, with the following properties: (i) The values of the function and the specified derivatives are reproduced exactly on the grid points, and (ii) the values of the function and the specified derivatives change continuously as the interpolating point crosses from one grid square to another.

It is important to understand that nothing in the equations of bicubic interpolation requires you to specify the extra derivatives *correctly*! The smoothness properties are tautologically "forced," and have nothing to do with the "accuracy" of the specified derivatives. It is a separate problem for you to decide how to obtain the values that are specified. The better you do, the more *accurate* the interpolation will be. But it will be *smooth* no matter what you do.

Best of all is to know the derivatives analytically, or to be able to compute them accurately by numerical means, at the grid points. Next best is to determine them by numerical differencing from the functional values already tabulated on the grid. The relevant code would be something like this (using centered differencing):

```
y1a[j][k]=(ya[j+1][k]-ya[j-1][k])/(x1a[j+1]-x1a[j-1]);
y2a[j][k]=(ya[j][k+1]-ya[j][k-1])/(x2a[k+1]-x2a[k-1]);
y12a[j][k]=(ya[j+1][k+1]-ya[j+1][k-1]-ya[j-1][k+1]+ya[j-1][k-1])
        /((x1a[j+1]-x1a[j-1])*(x2a[k+1]-x2a[k-1]));
```

To do a bicubic interpolation within a grid square, given the function y and the derivatives y1, y2, y12 at each of the four corners of the square, there are two steps: First obtain the 16 quantities c_{ij}, $i, j = 0, \ldots, 3$ using the routine bcucof below. (The formulas that obtain the c's from the function and derivative values are just a complicated linear transformation, with coefficients that, having been determined once in the mists of numerical history, can be tabulated and forgotten.) Next, substitute the c's into any or all of the following bicubic formulas for function and derivatives, as desired:

$$\begin{aligned} y(x_1, x_2) &= \sum_{i=0}^{3}\sum_{j=0}^{3} c_{ij} t^i u^j \\ y_{,1}(x_1, x_2) &= \sum_{i=0}^{3}\sum_{j=0}^{3} i c_{ij} t^{i-1} u^j (dt/dx_1) \\ y_{,2}(x_1, x_2) &= \sum_{i=0}^{3}\sum_{j=0}^{3} j c_{ij} t^i u^{j-1} (du/dx_2) \\ y_{,12}(x_1, x_2) &= \sum_{i=0}^{3}\sum_{j=0}^{3} ij c_{ij} t^{i-1} u^{j-1} (dt/dx_1)(du/dx_2) \end{aligned} \tag{3.6.6}$$

where t and u are again given by equation (3.6.4).

```
void bcucof(VecDoub_I &y, VecDoub_I &y1, VecDoub_I &y2, VecDoub_I &y12,
    const Doub d1, const Doub d2, MatDoub_O &c) {
```
interp_2d.h

Given arrays y[0..3], y1[0..3], y2[0..3], and y12[0..3], containing the function, gradients, and cross-derivative at the four grid points of a rectangular grid cell (numbered counterclockwise from the lower left), and given d1 and d2, the length of the grid cell in the 1 and 2 directions, this routine returns the table c[0..3][0..3] that is used by routine bcuint for bicubic interpolation.

```
    static Int wt_d[16*16]=
        {1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
        -3, 0, 0, 3, 0, 0, 0, 0,-2, 0, 0,-1, 0, 0, 0, 0,
        2, 0, 0,-2, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
        0, 0, 0, 0,-3, 0, 0, 3, 0, 0, 0, 0,-2, 0, 0,-1,
        0, 0, 0, 0, 2, 0, 0,-2, 0, 0, 0, 0, 1, 0, 0, 1,
        -3, 3, 0, 0,-2,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0,-3, 3, 0, 0,-2,-1, 0, 0,
        9,-9, 9,-9, 6, 3,-3,-6, 6,-6,-3, 3, 4, 2, 1, 2,
        -6, 6,-6, 6,-4,-2, 2, 4,-3, 3, 3,-3,-2,-1,-1,-2,
        2,-2, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 2,-2, 0, 0, 1, 1, 0, 0,
        -6, 6,-6, 6,-3,-3, 3, 3,-4, 4, 2,-2,-2,-2,-1,-1,
        4,-4, 4,-4, 2, 2,-2,-2, 2,-2,-2, 2, 1, 1, 1, 1};
    Int l,k,j,i;
    Doub xx,d1d2=d1*d2;
    VecDoub cl(16),x(16);
    static MatInt wt(16,16,wt_d);
    for (i=0;i<4;i++) {
```

Pack a temporary vector x.